Assignment 3 – CO3

Constructors, Destructors and Operator Overloading (Individual, 50 Marks)

Name: TELAPROLU ROHITA SAI
Register Number: 192425041

Q1 – Constructors and Constructor Overloading

Implementation (Task01/q1.cpp)

```
#include <iostream>
#include <string>
class Book {
    std::string title, author;
    int copies;
    static int ok(int c) { return c < 0 ? 0 : c; }   // single validation point
public:
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(ok(c)) {}       // main constructor
    explicit Book(const std::string& t) : Book(t, "Unknown", 1) {}  // donated
    Book() : Book("Untitled", "Unknown", 0) {}        // empty record
    void print() const {
        std::cout << title << " | " << author << " | " << copies << '\n';
    }
};
int main() {
    Book a("C++ Primer", "Lippman", 5), b("Donated Book"), c, d("Bad", "X", -7);
    a.print(); b.print(); c.print(); d.print();
}
```

Output:

```
C++ Primer | Lippman | 5
Donated Book | Unknown | 1
Untitled | Unknown | 0
Bad | X | 0
```

Design & Analysis

- **Compiler default constructor:** supplied only if the class declares no constructor at all. As soon as any constructor is written (e.g. Book(string)), the compiler does not generate Book(), so it must be written explicitly (done above).
- **Initializer list vs assignment:** members are initialised directly with the right value (one step). Assignment in the body first default-constructs the member and then assigns (two steps). It is also the only way to initialise const members, references and base classes.
- **Order of initialisation:** always the order of declaration in the class, not the order in the list. Writing the list in a different order only produces a -Wreorder warning and can use an uninitialised member.
- **Delegation:** Book(string) and Book() both call the main constructor, so the copies check (ok(c)) exists in only one place; no Book can hold a negative count.

```
Object states after construction
 a : title="C++ Primer", author="Lippman", copies=5
 b : title="Donated Book", author="Unknown", copies=1
 c : title="Untitled",    author="Unknown", copies=0
 d : title="Bad",         author="X",       copies=0   (-7 corrected)
```

Experimental Investigation

- **All constructors tested** – output above. Negative copies (-7) became 0.
- **Array without default constructor** (struct B { B(int){} }; B arr[3];) → error: no matching function for call to 'B::B()'.
- **Wrong initialisation order** (struct P { int b; int a; P() : a(1), b(a) {} };) → warning: 'P::a' will be initialized after 'int P::b' [-Wreorder]; b receives an uninitialised value.

Design Question – explicit

Mark a single-argument constructor explicit when an automatic conversion would be surprising or wrong (a string silently becoming a Book).

```
struct B { std::string t; explicit B(const std::string& s) : t(s) {} };
void f(B) {}
B x = std::string("a");   // OK without explicit, ERROR with explicit
f(std::string("a"));      // OK without explicit, ERROR with explicit
B y("a");                 // always OK (direct initialisation)
```

With explicit the compiler reports: conversion from 'std::string' to non-scalar type 'B' requested.

Alternative Concept and Complexity

```
Book(std::string t = "", std::string a = "Unknown", int copies = 0)
    : title(t), author(a), copies(copies < 0 ? 0 : copies) {}
```

One constructor with default arguments replaces three constructors. Time and space of creation are O(1) in both designs. The default-argument version checks copies in exactly one place without delegation, but it cannot give different defaults to different situations (e.g. a donated book has 1 copy, an empty record has 0) and allows odd calls such as Book("", "", 5). The overload + delegation design is clearer; the default-argument design is shorter.

Q2 – Copy Constructors: Shallow and Deep Copies

Version 1 – without copy constructor (q2_shallow.cpp)

```
#include <iostream>
#include <string>
class Prescription {
    int n; std::string* medicines;
public:
    explicit Prescription(int c) : n(c), medicines(new std::string[c]) {}
    ~Prescription() { delete[] medicines; }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2); p.set(0, "Paracetamol");
    Prescription q = p;           // shallow copy
    q.set(0, "Cetirizine");
    std::cout << p.get(0) << '\n'; // original changed!
}
```

Result: p.get(0) prints "Cetirizine" – the original changed because p and q share one array. At program end both destructors delete[] the same memory (double delete). AddressSanitizer reports: ERROR: AddressSanitizer: heap-use-after-free … in Prescription::~Prescription().

Version 2 – complete code (q2.cpp)

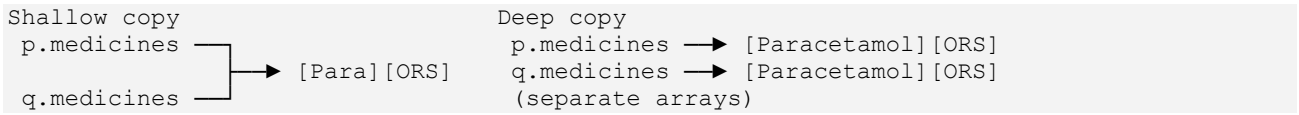
```
#include <iostream>
#include <string>
class Prescription {
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
    Prescription(const Prescription& o) : n(o.n), medicines(new std::string[o.n]) {
        for (int i = 0; i < n; i++) medicines[i] = o.medicines[i];
    }
    Prescription& operator=(const Prescription& o) {
        if (this != &o) {
            std::string* t = new std::string[o.n];
            for (int i = 0; i < o.n; i++) t[i] = o.medicines[i];
            delete[] medicines;
            medicines = t; n = o.n;
        }
        return *this;
    }
    ~Prescription() { delete[] medicines; }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol"); p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
              << " | " << r.get(0) << ' ' << r.get(1) << '\n';
    p = p;   // self-assignment test
    std::cout << p.get(0) << '\n';
}
```

Output:

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
Ibuprofen
```

The output matches the expected output, and the sanitizer reports no error. Deleting the copy constructor again gives the same heap-use-after-free (double delete) error as Version 1.

Design & Analysis



- **Double delete:** p and q hold the same pointer; the first destructor frees the array, the second frees it again, which is undefined behaviour (crash).
- **Rule of Three:** if a class needs a user-written destructor, copy constructor or copy assignment operator, it almost certainly needs all three, because all three deal with the ownership of the same resource.
- **Copy constructor is called when:** (1) an object is initialised from another (Prescription q = p;), (2) an object is passed to a function by value, (3) an object is returned by value (when not elided).

Experimental Investigation

- **Copy count:** with a message in the copy constructor, passing by value prints one "copy" per call; passing by const reference prints none (tested: byVal(p) → 1 copy, byRef(p) → 0, P q = p → 1).
- **Self-assignment:** p = p is safe because of the this != &o guard; without it the array would be deleted before being copied.

Medicines (n)	Deep copy time (g++ -O0, my run)
10	≈ 17 µs
1,000	≈ 17 µs
100,000	≈ 2,490 µs (2.5 ms)

Time grows linearly with n for large n (small sizes are dominated by allocation cost). Your values will differ with your machine.

Conclusion

The compiler-generated copy constructor is enough when the class holds only values or objects that copy themselves correctly (int, std::string, std::vector) – Rule of Zero. You must write your own when the class owns a raw resource (new[], file handle, socket) where a member-wise copy would copy only the pointer.

Alternative Concept and Complexity

```
class Prescription {
    std::vector<std::string> medicines;
public:
    explicit Prescription(int n) : medicines(n) {}
    // no copy constructor, assignment or destructor needed
};
```

The vector version needs none of the three functions. Copying n medicines is O(n) time and O(n) space in both the vector and the raw array designs; a shallow copy is only O(1) time and space but is unsafe because of the double delete.

Q3 – Dynamic Constructors and Destructors

Implementation (q3.cpp)

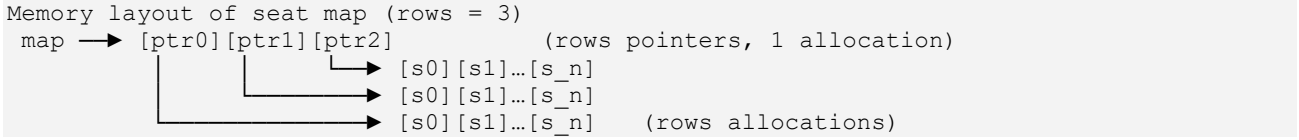
```
#include <iostream>
class Event {
    int rows, seats, booked = 0;
    char** map;
public:
    Event(int r, int s) : rows(r), seats(s), map(new char*[r]) {
        for (int i = 0; i < r; i++) map[i] = new char[s]();   // '\0' = free
        std::cout << "Event " << r << "x" << s << " created\n";
    }
    bool book(int r, int s)   { if (r<0||r>=rows||s<0||s>=seats||map[r][s]) return false;
map[r][s]=1; booked++; return true; }
    bool cancel(int r, int s) { if (r<0||r>=rows||s<0||s>=seats||!map[r][s]) return
false; map[r][s]=0; booked--; return true; }
    ~Event() {
        for (int i = 0; i < rows; i++) delete[] map[i];
        delete[] map;
        std::cout << "Event closed: " << booked << "/" << rows * seats << " booked\n";
    }
};
int main() {
    Event e1(2, 3);                         // stack
    e1.book(0, 1); e1.book(1, 2); e1.cancel(0, 1);
    { Event e2(1, 1); e2.book(0, 0); }    // block scope
    Event* e3 = new Event(2, 2);           // heap
    e3->book(1, 1);
    delete e3;
    Event e0(0, 5);                         // zero rows test
}
```

Output:

```
Event 2x3 created
Event 1x1 created
Event closed: 1/1 booked      (block ends)
Event 2x2 created
Event closed: 1/4 booked      (delete e3)
Event 0x5 created             (zero rows works, no seats)
Event closed: 0/0 booked      (e0, last created, first destroyed)
Event closed: 1/6 booked      (e1)
```

Design & Analysis

- **Order:** objects are destroyed in reverse order of construction. A block-scope object is destroyed at the closing brace; a heap object only when delete is called; stack objects at the end of main.
- **Forgotten delete:** the destructor never runs, so memory leaks (AddressSanitizer: LeakSanitizer: detected memory leaks – 44 bytes leaked in 4 allocations in my test). Using delete instead of delete[] on an array is undefined behaviour (AddressSanitizer: alloc-dealloc-mismatch (operator new [] vs operator delete)).
- **Destructor takes no arguments and cannot be overloaded:** it is called automatically at scope end with no way to pass arguments, and overloading would make it ambiguous which destructor to call. There is exactly one per class.



Experimental Investigation

Seat map	Seats	Memory for seats (char)	Plus row pointers (8 B each)
10 × 20	200	200 B	80 B
100 × 200	20,000	≈ 20 KB	800 B
1,000 × 2,000	2,000,000	≈ 2 MB	8 KB

- **Missing delete:** leak report (above); with correct code the sanitizer is clean.
- **Zero rows:** works – new char*[0] is valid, the loop runs zero times and the destructor prints 0/0.

Conclusion – RAII

When release is tied to the destructor, memory is freed on every exit path (normal end, early return, exception) without the programmer remembering to call a cleanup function, so leaks and double frees become far less likely.

Predict the Output (q3p.cpp)

My prediction (written before running):

```
+A +B -B copyAc use -Ac +P -P end -A
```

Actual output:

```
+A +B -B copyAc use -Ac +P -P end -A
```

No difference. Explanation: +A (a created); +B then -B (b destroyed at the closing brace); use(a) takes Logger by value, so the copy constructor builds a temporary named "Ac" (copyAc), "use" prints, and the copy is destroyed when the function returns (-Ac); +P and -P for the heap object (delete); "end" is printed; finally a is destroyed at the end of main (-A).

Alternative Concept and Complexity

```
std::vector<char> map(rows * seats);          // one block
map[r * seats + s] = 1;                       // index r*seats+s
```

Access time is O(1) in all designs. The pointer-array design makes rows + 1 allocations and stores rows extra pointers (8·rows bytes); the single-block (or a flat vector) design makes 1 allocation, has no pointer overhead, better cache locality, and needs no loops in the destructor. The vector<vector<char>> version is easy and safe but still has rows + 1 allocations.

Q4 – Overloading Arithmetic and Stream Operators

Implementation (q4.cpp)

```
#include <iomanip>
#include <iostream>
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    Money operator+(const Money& o) const { Money m; m.paise = paise + o.paise; return
m; }
    Money operator-(const Money& o) const { Money m; m.paise = paise - o.paise; return
m; }
    Money& operator+=(const Money& o) { paise += o.paise; return *this; }
    bool operator==(const Money& o) const { return paise == o.paise; }
    bool operator<(const Money& o) const { return paise < o.paise; }
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long a = m.paise < 0 ? -m.paise : m.paise;
        return os << (m.paise < 0 ? "-" : "") << "Rs " << a / 100 << '.'
                  << std::setw(2) << std::setfill('0') << a % 100 << std::setfill(' ');
    }
    friend std::istream& operator>>(std::istream& is, Money& m) {
        long long r; int p; is >> r >> p; m = Money(r, p); return is;
    }
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
    std::cout << Money(0, 75) + Money(0, 50) << '\n';   // carry test
    std::cout << Money(10) - Money(12, 50) << '\n';     // negative test
    Money t; t += Money(5, 5); t += Money(1, 96); std::cout << t << ' ' << (a < b) <<
'\n';
}
```

Output:

```
Rs 200.00
true
Rs 1.25        (0.75 + 0.50, paise carried into rupees)
-Rs 2.50        (10 - 12.50, negative amount)
Rs 7.01 false
```

Design & Analysis

- **+ vs +=:** + creates and returns a new object and leaves both operands unchanged (like a + b for ints). += modifies the left operand and returns Money& (*this) so that calls can be chained: (a += b) += c.
- **Members:** +, -, +=, ==, < are members because the left operand is a Money. **Friends:** << and >> have a stream on the left.
- **Why << is a non-member:** a member operator<< would need the left operand to be a Money (m << cout). The left operand is std::ostream, a class we cannot modify, so it must be a free (friend) function to access the private paise.
- **Compiler translation:** a + b becomes a.operator+(b); cout << c becomes operator<<(cout, c).
- **Negative amounts:** allowed (balance or refund) and printed with a leading "-" sign by <<.

Experimental Investigation

- **Readability:** Money total = price + tax - discount; is easier to read than total = subtract(add(price, tax), discount); and cout << total; replaces print(total);.

- **double vs Money:** adding 0.10 one million times in a double gives 100000.00000133… (representation error), while Money (integer paise) gives exactly Rs 100000.00. Measure with std::chrono as in Q2.

Engineering Decision

Multiplying Money by Money gives "rupees squared", which has no meaning, so it should not be provided. Sensible operators: +, -, +=, -=, ==, <, and multiplication/division by a plain number (Money * double for quantity, tax or discount). Money / Money may return a plain ratio.

Alternative Concept and Complexity

```
friend Money operator+(const Money& a, const Money& b)   // non-member form
{ return Money(0, 0) /* ... */; }
```

Time and space are O(1) for member, friend and named functions (add(), equals()) alike. A member operator+ needs a Money on the left, so m + 100 works (100 converts implicitly through the constructor) but 100 + m fails. A non-member (friend) operator+ lets both 100 + m and m + 100 compile because implicit conversion applies to both arguments.

Q5 – Overloading Increment, Comparison and Subscript Operators

Implementation (q5.cpp)

```
#include <iostream>
#include <string>
#include <vector>
class Time {
    int s;   // total seconds
public:
    Time(int h = 0, int m = 0, int sec = 0) : s(h * 3600 + m * 60 + sec) {}
    Time& operator++()   { ++s; return *this; }            // prefix
    Time  operator++(int) { Time old = *this; ++s; return old; } // postfix
    bool operator==(const Time& o) const { return s == o.s; }
    bool operator<(const Time& o) const { return s < o.s; }
    friend std::ostream& operator<<(std::ostream& os, const Time& t) {
        return os << t.s / 3600 << ':' << (t.s / 60) % 60 / 10 << (t.s / 60) % 60 % 10
                  << ':' << t.s % 60 / 10 << t.s % 60 % 10;
    }
};
class Playlist {
    std::vector<std::string> t;
    std::string bad = "<invalid>";
public:
    Playlist(std::vector<std::string> v) : t(v) {}
    std::string& operator[](int i)             { if (i < 0 || i >= (int)t.size())
{ std::cerr << "index error\n"; return bad; } return t[i]; }
    const std::string& operator[](int i) const { if (i < 0 || i >= (int)t.size())
{ std::cerr << "index error\n"; return bad; } return t[i]; }
};
int main() {
    Time t(0, 59, 58), end(1, 0, 1);
    Time a = t++, b = ++t;
    std::cout << a << ' ' << b << ' ' << t << '\n';   // 0:59:58 1:00:00 1:00:00
    Playlist p({"Song A", "Song B"});
    for (int i = 0; i < 2; i++) std::cout << p[i] << '\n';
    p[-1]; p[2];
    while (t < end) ++t;
    std::cout << t << ' ' << (t == end) << '\n';
}
```

Output:

```
0:59:58 1:00:00 1:00:00
Song A
Song B
1:00:01 1
(stderr) index error
(stderr) index error
```

Design & Analysis

- **Prefix vs postfix:** the compiler distinguishes them by a dummy int parameter: operator++() is prefix, operator++(int) is postfix (the int is unused).
- **Postfix returns a copy:** it must return the value before the increment, so it saves the old object, increments, and returns the saved copy – an extra copy makes it slower than prefix.
- **const and non-const operator[]:** the non-const version returns a reference that can be modified (p[0] = "X"); the const version is needed so that a const Playlist can be read, and it returns a const reference.
- **Out-of-range index:** must never read memory outside the container; here an error is printed and a safe dummy is returned (alternatively throw std::out_of_range).

Experimental Investigation

- **a = t++, b = ++t** with t = 0:59:58: a = 0:59:58 (old value), t becomes 0:59:59, then ++t makes 1:00:00 so b = 1:00:00 and t = 1:00:00. This also tests the roll-over 59:59 → 1:00:00.

- **playlist[-1] and playlist[size]:** both print "index error".

10,000,000 increments (my run)	-O0	-O2
Prefix ++	≈ 10.9 ms	≈ 0.01 ms (optimised away)
Postfix ++	≈ 18.0 ms	≈ 0.0002 ms (optimised away)

Conclusion

For class types ++i is preferred because it avoids creating a temporary copy; for simple types the optimiser removes the difference, but for large objects (iterators, big numbers) the copy can be expensive.

Predict the Output (q5p.cpp)

My prediction (before running): 5 7 7

Actual output:

```
5 7 7
```

c++ returns a copy of the old value (5) then c = 6; ++c increments c to 7 and returns a reference, so b = 7 and c = 7. The postfix version makes a copy of the object (Counter old = *this).

Alternative Concept and Complexity

```
std::string& at(int i) { if (i < 0 || i >= (int)t.size()) throw std::out_of_range("bad index"); return t[i]; }
```

at(i) throws an exception (the caller must handle it); an unchecked operator[] has no check (fastest but undefined behaviour if wrong); the range-checked operator[] prints an error. Each access is O(1) in all designs; the checked versions add one comparison, which is negligible (a few ms for 10,000,000 accesses – measure with std::chrono). Space is O(1).

Q6 – The Rules of Operator Overloading

Implementation (q6.cpp)

```
#include <cstring>
#include <iostream>
class Text {
    char* s;
public:
    Text(const char* c = "") : s(std::strcpy(new char[std::strlen(c) + 1], c)) {}
    Text(const Text& o) : Text(o.s) {}
    Text& operator=(const Text& o) {
        if (this != &o) { char* t = std::strcpy(new char[std::strlen(o.s) + 1], o.s);
delete[] s; s = t; }
        return *this;
    }
    ~Text() { delete[] s; }
    Text operator+(const Text& o) const {
        Text r;
        delete[] r.s;
        r.s = new char[std::strlen(s) + std::strlen(o.s) + 1];
        std::strcpy(r.s, s); std::strcat(r.s, o.s);
        return r;
    }
    bool operator==(const Text& o) const { return !std::strcmp(s, o.s); }
    bool operator!=(const Text& o) const { return !(*this == o); }
    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os <<
t.s; }
};
int main() {
    Text a("Hello"), b(", "), c("World"), e;
    Text d = a + b + c;
    std::cout << d << '\n' << (d == Text("Hello, World")) << (d != a) << '\n';
    d = d;                                  // self-assignment
    std::cout << d << '[' << e << ']' << '\n';   // empty message
    Text big(std::string(100000, 'x').c_str());
    std::cout << (big + big + big == big + big + big) << '\n';
}
```

Output:

```
Hello, World
11
Hello, World[]
1
```

The program runs clean under -fsanitize=address (no errors). The output shows concatenation, == and != (printed as 1 and 1), self-assignment (d = d), an empty message, and a 100,000-character message.

Design & Analysis

Operator	Why it cannot be overloaded
::	Scope resolution; works on names (classes/namespaces), not objects
.	Member access must always have a fixed meaning; the language needs it to reach members
.*	Pointer-to-member access; same reason as .
?:	Cannot pass all three operands with short-circuit evaluation to a function
sizeof	Evaluated at compile time; its result is needed by the compiler itself

- **Rules:** precedence, associativity and arity (number of operands) cannot be changed, new operators cannot be invented, and at least one operand must be a user-defined type (int + int cannot be redefined).

- **operator= must be a member:** the language requires =, [], () and -> to be members so that the left operand is always the object being modified and the compiler does not generate a conflicting default assignment.
- **Use a friend** when the operator needs private data and the left operand is not the class (e.g. cout << t), or when symmetric conversion is needed (100 + m).

Experimental Investigation

- **Copies for a + b + c:** (a + b) creates one temporary, then (temp + c) creates a second one; so two new Text objects are built (and copied by the copy constructor if no move/elision is applied). Add a message in the copy constructor to count them.
- **Self-assignment:** d = d is safe due to the this != &o check; without it, delete[] s would destroy the data before copying.
- **Versus std::string:** std::string is faster/equally fast for 100,000 concatenations because of capacity growth, move semantics and small-string optimisation; Text copies all characters every time (O(total length)).

Conclusion

Writing Text teaches ownership, deep copying, the Rule of Three, and how operators map to function calls. In real programs std::string should be used; a custom string class is still justified for learning, for special memory control (embedded systems, custom allocators, fixed buffers) or for a specific storage format.

Alternative Concept and Complexity

```
Text& operator+=(const Text& o);   // append in place
Text operator+(Text a, const Text& b) { a += b; return a; }
```

Both versions copy characters: evaluating a + b + c copies n1 + n2 characters for the first temporary and (n1 + n2) + n3 for the second, so the total is O(2·n1 + 2·n2 + n3), i.e. O(n1 + n2 + n3). The "+= on a copy" version takes its first argument by value (so it can be moved/elided) and builds only one buffer per +, giving the same Big-O with less temporary space; std::string reaches the same time but with move semantics and amortised capacity growth the temporary space is smaller still.

Q7 – Complex-Number Class from First Principles (AC Impedance)

Stage 1 – Interpret the Problem

- **Data:** real part (resistance R) and imaginary part (reactance X) – two doubles. Resistor = R + 0j, inductor = 0 + jωL, capacitor = 0 − j/(ωC).
- **Operations:** add (series), multiply and divide (parallel = Z1·Z2/(Z1+Z2)), subtract, compare, print in rectangular form, magnitude |Z|.
- **Constructors:** default (0 + 0j), from real part only (resistor), from real and imaginary parts (inductor/capacitor).
- **Measurable requirements:** (1) Z1 + Z2 differs from hand calculation by < 1e-9; (2) series RLC (R=100 Ω, L=0.1 H, C=1 μF, ω=1000 rad/s) gives 100 − j900; (3) parallel R and L gives 50 + j50; (4) dividing by 0 + 0j is detected and reported without crashing.

Stage 2 – Design

Operator	Formula	Form
a + b	(a + c) + j(b + d)	member
a - b	(a − c) + j(b − d)	member
a * b	(ac − bd) + j(ad + bc)	member
a / b	((ac + bd) + j(bc − ad)) / (c² + d²)	member, throws if c² + d² = 0
==	|re1 − re2| < 1e-9 and |im1 − im2| < 1e-9	member
<<	prints "re + j im"	friend (stream on left)
double + Complex	converted by Complex(double)	friend (symmetry)

Stage 3 – Implement and Verify (q7.cpp)

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}
    Complex(double r) : re(r), im(0) {}
    Complex(double r, double i) : re(r), im(i) {}
    double mag() const { return std::sqrt(re * re + im * im); }
    Complex operator+(const Complex& o) const { return Complex(re + o.re, im + o.im); }
    Complex operator-(const Complex& o) const { return Complex(re - o.re, im - o.im); }
    Complex operator*(const Complex& o) const { return Complex(re * o.re - im * o.im, re
* o.im + im * o.re); }
    Complex operator/(const Complex& o) const {
        double d = o.re * o.re + o.im * o.im;
        if (d == 0) throw std::runtime_error("division by zero impedance");
        return Complex((re * o.re + im * o.im) / d, (im * o.re - re * o.im) / d);
    }
    bool operator==(const Complex& o) const { return std::fabs(re - o.re) < 1e-9 &&
std::fabs(im - o.im) < 1e-9; }
    friend Complex operator+(double a, const Complex& b) { return Complex(a) + b; }
    friend std::ostream& operator<<(std::ostream& os, const Complex& c) {
        return os << c.re << (c.im < 0 ? " - j" : " + j") << std::fabs(c.im);
    }
};
int main() {
    double w = 1000, L = 0.1, C = 1e-6, R = 100;
    Complex ZR(R), ZL(0, w * L), ZC(0, -1 / (w * C));
    Complex series = ZR + ZL + ZC;
    Complex par = (ZR * ZL) / (ZR + ZL);
    std::cout << "Series   : " << series << "  |Z| = " << series.mag() << '\n';
    std::cout << "Parallel : " << par << "  |Z| = " << par.mag() << '\n';
    try { std::cout << series / Complex(0, 0); }
```

```
    catch (const std::exception& e) { std::cout << "Error: " << e.what() << '\n'; }
}
```

Output:

```
Series   : 100 - j900  |Z| = 905.539
Parallel : 50 + j50  |Z| = 70.7107
Error: division by zero impedance
```

Hand check – series: ZL = j(1000·0.1) = j100, ZC = −j/(1000·1e-6) = −j1000, so Z = 100 + j100 − j1000 = 100 − j900, |Z| = √(100² + 900²) = 905.54 ✓.

Hand check – parallel (R with L): (100·j100)/(100 + j100) = j10000(100 − j100)/20000 = (10⁶ + j10⁶)/20000 = 50 + j50, |Z| = 70.71 ✓.

Division by zero: dividing by 0 + 0j throws std::runtime_error, caught in main and reported.

Stage 4 – Defend the Design

- **Versus std::complex<double>:** std::complex is tested, optimised, has abs, arg, polar, sqrt and many more functions; mine is shorter and teaches the idea, and I choose how to handle division by zero (exception) whereas std::complex returns inf/NaN.
- **Why each constructor:** default for arrays and uninitialised values; Complex(double) for resistors and implicit conversion (5 + z); Complex(r, i) for inductors and capacitors.
- **Why each operator:** + series; * and / parallel formula; − differences/voltage drops; == testing/verification; << printing results.
- **Limitation:** works in double precision only (rounding errors; == needs a tolerance), no polar form, and no functions such as sqrt or exp, no unit/frequency tracking.

Alternative Concept and Complexity

```
#include <complex>
std::complex<double> ZR(100, 0), ZL(0, 100), par = ZR * ZL / (ZR + ZL);
std::cout << par << " |Z|=" << std::abs(par);   // prints (50,50)
```

Time per operation is O(1) and space is two doubles in both designs. std::complex is more accurate for division and magnitude (it uses scaling to avoid overflow, std::abs uses hypot), whereas my formulas can overflow or lose precision for very large or very small values. For division by zero std::complex silently gives inf/NaN so the programmer must check the divisor manually, while my class throws a clear error.